THE UNIVERSITY OF ALABAMA®

WHERE LEGENDS ARE MADE®

Non-Binary (N-ary) Trees

N-ary Trees and Traversals

CS 201: Data Structures and Algorithms
Jiacheng Li, Ph.D.

N-ary Tree

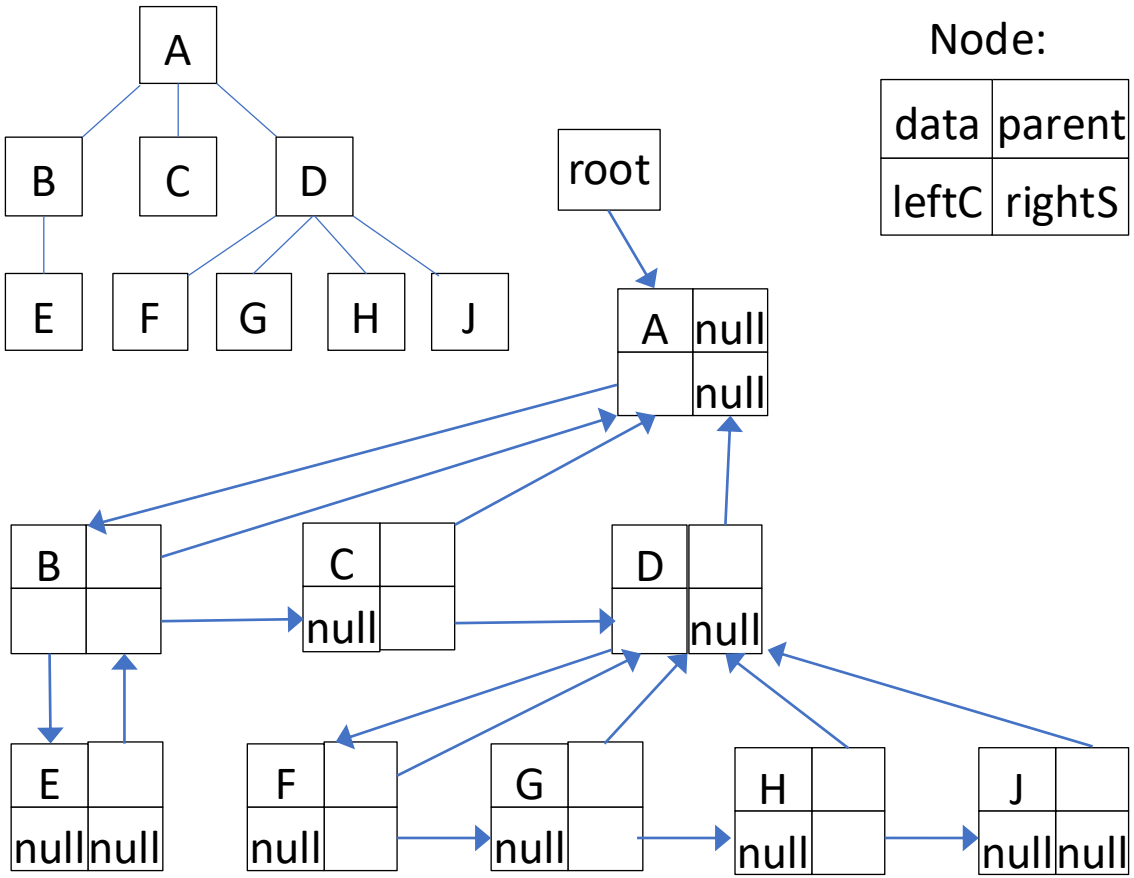
- A *Nonbinary Tree*, also known as an **N-ary Tree**, is a tree data structure in which each node can have zero or more child nodes, up to a maximum of N children.
- An N-ary Tree is defined as a rooted tree in which each node has at most N children.
  - When $N = 2$, the structure becomes a binary tree.
  - When $N > 2$, it is called a non-binary or N-ary tree.

N-ary Tree Programmatically

```
class Node {
  ElementType data;
  Node parent, leftmostChild, rightSibling;
  // singly-linked list of each node's children
}

class Tree {
  Node root;
}
```

N-ary Tree Visually

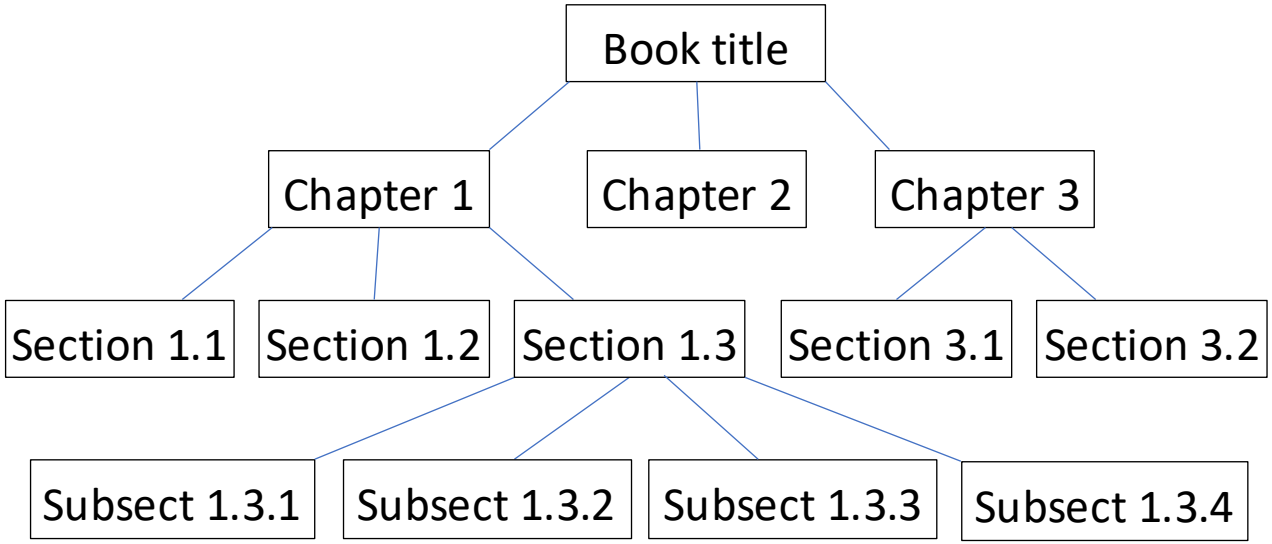


Alternative N-ary Tree

```
class Node {
  ElementType data;
  Node parent, leftmostChild, rightmostChild,
       leftSibling, rightSibling;
  // doubly-linked list of each node's children
}
class Tree {
  Node root;
}
```

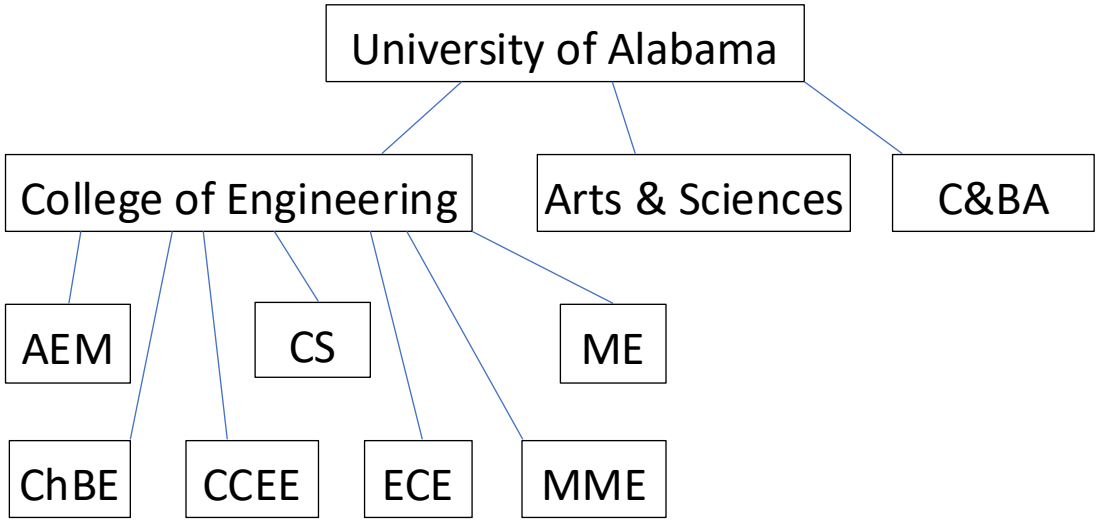
Applications

- Contents of a book



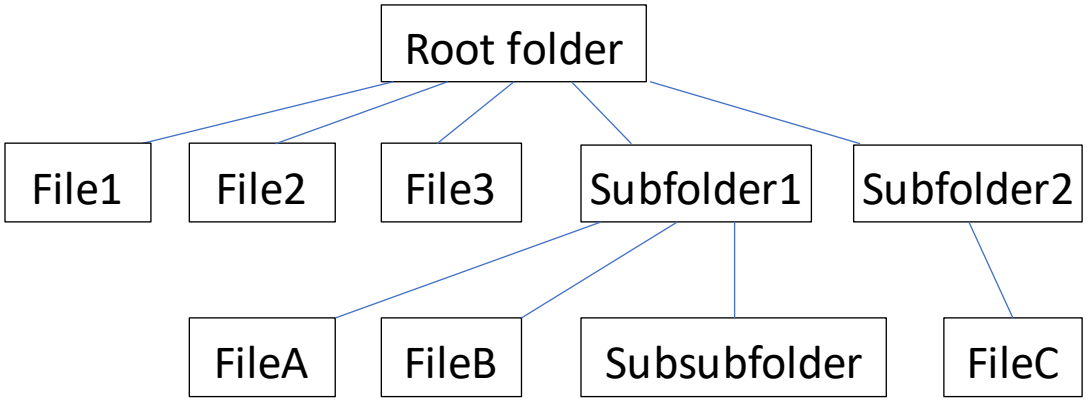
Applications (cont.)

- Hierarchy of an organization



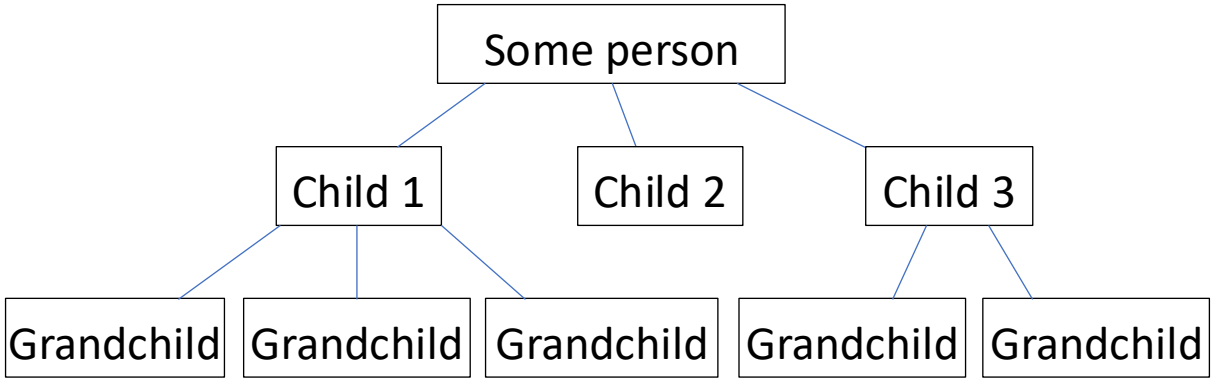
Applications (cont.)

- Filesystem



Applications (cont.)

- Family tree (descendants)



Traversals

- Pre-order
- Post-order
- Level-order

- No natural definition of inorder for n-ary trees

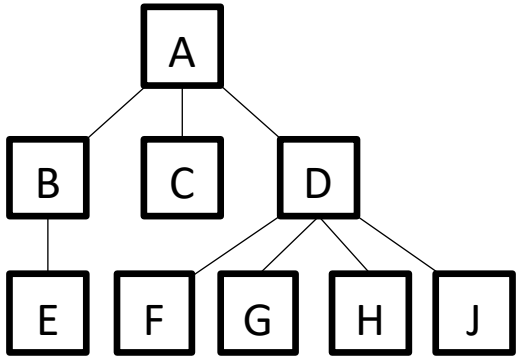
Preorder Traversal (CLR)

```
void preorder(Tree T) {
  preorder(T.root);       // a wrapper function
}

void preorder(Node p) {
  if (p == null) return;
  visit(p); // Current
  preorder(p.leftmostChild); // Left
  preorder(p.rightSibling); // Right
}
```

Preorder Traversal

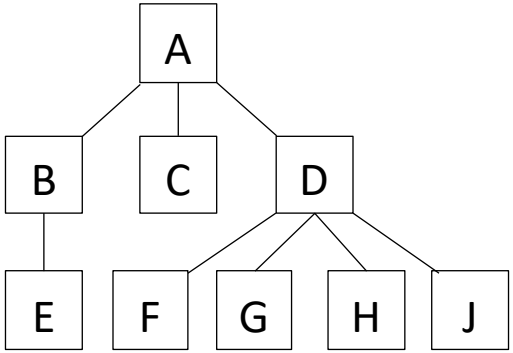
```
void preorder(Node p) {
  if (p == null) return;
  visit(p);
  preorder(p.leftmostChild);
  preorder(p.rightSibling);
}
```



Prefix: A B E C D F G H J

Preorder Traversal

```
void preorder(Node p) {
  if (p == null) return;
  visit(p);
  preorder(p.leftmostChild);
  preorder(p.rightSibling);
}
```



Prefix: A <u>B E</u> <u>C</u> <u>D F G H J</u>

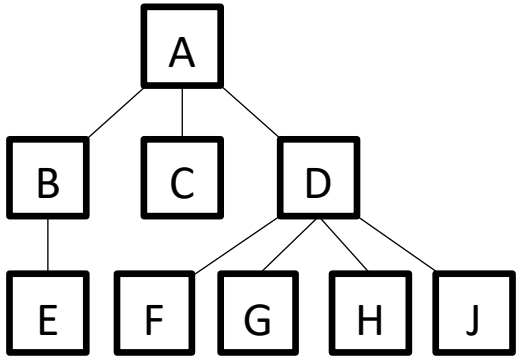
Postorder Traversal

```
void postorder(Tree T) {
  postorder(T.root);    // a wrapper function
}

void postorder(Node p) {
  if (p == null) return;
  postorder(p.leftmostChild);
  visit(p);
  postorder(p.rightSibling);
}
```

Postorder Traversal

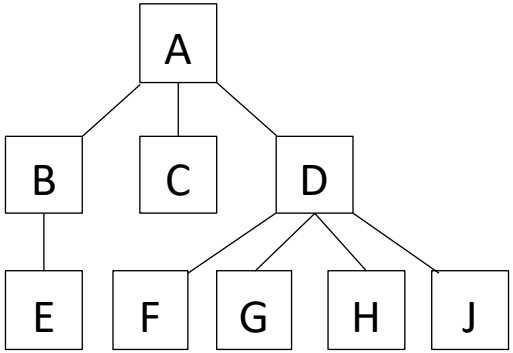
```
void postorder(Node p) {
  if (p == null) return;
  postorder(p.leftmostChild);
  visit(p);
  postorder(p.rightSibling);
}
```



Postfix: E B C F G H J D A

Postorder Traversal

```
void postorder(Node p) {
  if (p == null) return;
  postorder(p.leftmostChild);
  visit(p);
  postorder(p.rightSibling);
}
```



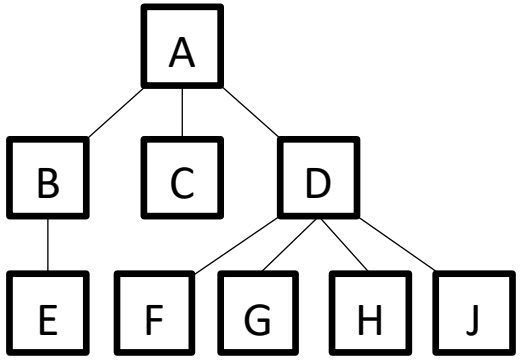
Postfix: <u>E B</u> <u>C</u> <u>F G H J D</u> A

Level-order Traversal

```
void levelOrder(Tree T) {
  Queue Q();   // FIFO
  Q.enqueue(T.root);
  while (not Q.isEmpty()) {
    Node p = Q.dequeue();
    visit(p);
    for (Node c = p.leftmostChild; c != null; c = c.rightSibling)
      Q.enqueue(c);
  }
}
```

Level-order Traversal

```
void levelOrder(Tree T) {
  Queue Q();
  Q.enqueue(T.root);
  while (not Q.isEmpty()) {
    Node p = Q.dequeue();
    visit(p);
    for (Node c = p.leftmostChild; c != null; c = c.rightSibling)
      Q.enqueue(c);
  }
}
```



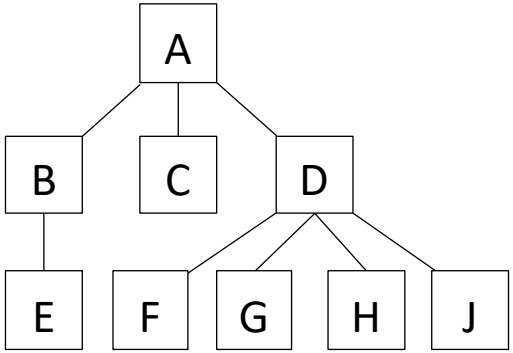
Level-order: A B C D E F G H J

Queue

A
B
C
D
E
F
G
H
J

Level-order Traversal

```
void levelOrder(Tree T) {
  Queue Q();
  Q.enqueue(T.root);
  while (not Q.isEmpty()) {
    Node p = Q.dequeue();
    visit(p);
    for (Node c = p.leftmostChild; c != null; c = c.rightSibling)
      Q.enqueue(c);
  }
}
```



Level-order: A <u>B C D</u> <u>E F G H J</u>

Analysis

- Let n = number of nodes in the tree.
- Each traversal spends $\theta(1)$ time at each node, so each traversal has $\theta(n)$ total running time.
- Same as for binary trees.

Questions?